TCP Chat Client and Server — Lab Exam Algorithms

TCP Chat Client — Algorithm

1. Start the client program.

2. Create a TCP socket and establish a connection with the server running on `localhost` at port `5555`.

```
Socket s = new Socket("localhost", 5555);
```

3. Create a `BufferedReader` object to receive messages from the server through the socket input stream.

```
BufferedReader in =
    new BufferedReader(
        new InputStreamReader(s.getInputStream()));
```

4. Create a `PrintWriter` object to send messages to the server through the socket output stream.

```
PrintWriter out =
    new PrintWriter(
        s.getOutputStream(), true);
```

5. Create a `BufferedReader` object to read messages entered by the user from the keyboard.

```
BufferedReader sc =
    new BufferedReader(
        new InputStreamReader(System.in));
```

6. Display a message instructing the user to type `"end"` to terminate the chat.

```
System.out.println("Type end to Quit:");
```

7. Repeat the following steps inside a loop: prompt the user to enter a message and read the message from the keyboard.

```
while (true)
{
    System.out.print("Message to Server: ");
    msg = sc.readLine();
```

8. Send the entered message to the server using `println()`.

```
out.println(msg);
```

9. Check whether the message is `"end"`. If it is `"end"`, terminate the loop.

```
if (msg.equals("end"))
    break;
```

10. Receive the reply from the server using `readLine()` and display it.

```
System.out.println("Server: " + in.readLine());
```

11. Close the client socket after the communication is completed.

```
s.close();
```

12. Stop the program.

TCP Chat Server — Algorithm

1. Start the server program.

2. Create a `ServerSocket` using port `5555` and wait for a client connection.

```
ServerSocket ss = new ServerSocket(5555);
```

3. Accept the client connection using `accept()`.

```
Socket s = ss.accept();
```

4. Display a message indicating that the server has started and the client is connected.

```
System.out.println("Server started");
System.out.println("Client connected");
```

5. Create a `BufferedReader` object to receive messages from the client.

```
BufferedReader in =
    new BufferedReader(
        new InputStreamReader(s.getInputStream()));
```

6. Create a `PrintWriter` object to send messages to the client.

```
PrintWriter out =
    new PrintWriter(
        s.getOutputStream(), true);
```

7. Create a `BufferedReader` object to read reply messages entered by the server user.

```
BufferedReader sc =
    new BufferedReader(
        new InputStreamReader(System.in));
```

8. Repeat the following steps inside a loop and receive a message from the client.

```
while (true)
{
    String msg = in.readLine();
```

9. Check whether the received message is `"end"`. If it is `"end"`, terminate the loop.

```
if (msg.equals("end"))
    break;
```

10. Display the message received from the client.

```
System.out.println("Client: " + msg);
```

11. Read a reply message from the server user and send it to the client.

```
System.out.print("Message to Client: ");
msg = sc.readLine();
out.println(msg);
```

12. Check whether the reply is `"end"`. If it is `"end"`, terminate the loop.

```
if (msg.equals("end"))
    break;
```

13. Display that the client has disconnected.

```
System.out.println("Client disconnected");
```

14. Close the client socket and server socket.

```
s.close();
ss.close();
```

15. Stop the program.